SHREYA CU2425OO18 6O

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from skimage.util import random_noise
from skimage.filters import gaussian, median
from scipy.ndimage import uniform_filter
from skimage.morphology import disk
```

1.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("/content/SCENERY.jpeg", 0)

# Add Salt & Pepper noise
noise = img.copy()
prob = 0.05

random = np.random.rand(*img.shape)
noise[random < prob/2] = 0       # Pepper
noise[random > 1-prob/2] = 255   # Salt

# Apply filters
mean = cv2.blur(noise, (5, 5))
gaussian = cv2.GaussianBlur(noise, (5, 5), 0)
median = cv2.medianBlur(noise, 5)

# Calculate MSE
def mse(original, filtered):
    return np.mean((original.astype(float) - filtered.astype(float)) ** 2)

print("MSE Results:")
print("Mean Filter     :", mse(img, mean))
print("Gaussian Filter :", mse(img, gaussian))
print("Median Filter   :", mse(img, median))

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(noise, cmap='gray')
plt.title("Salt & Pepper Noise")
plt.axis("off")

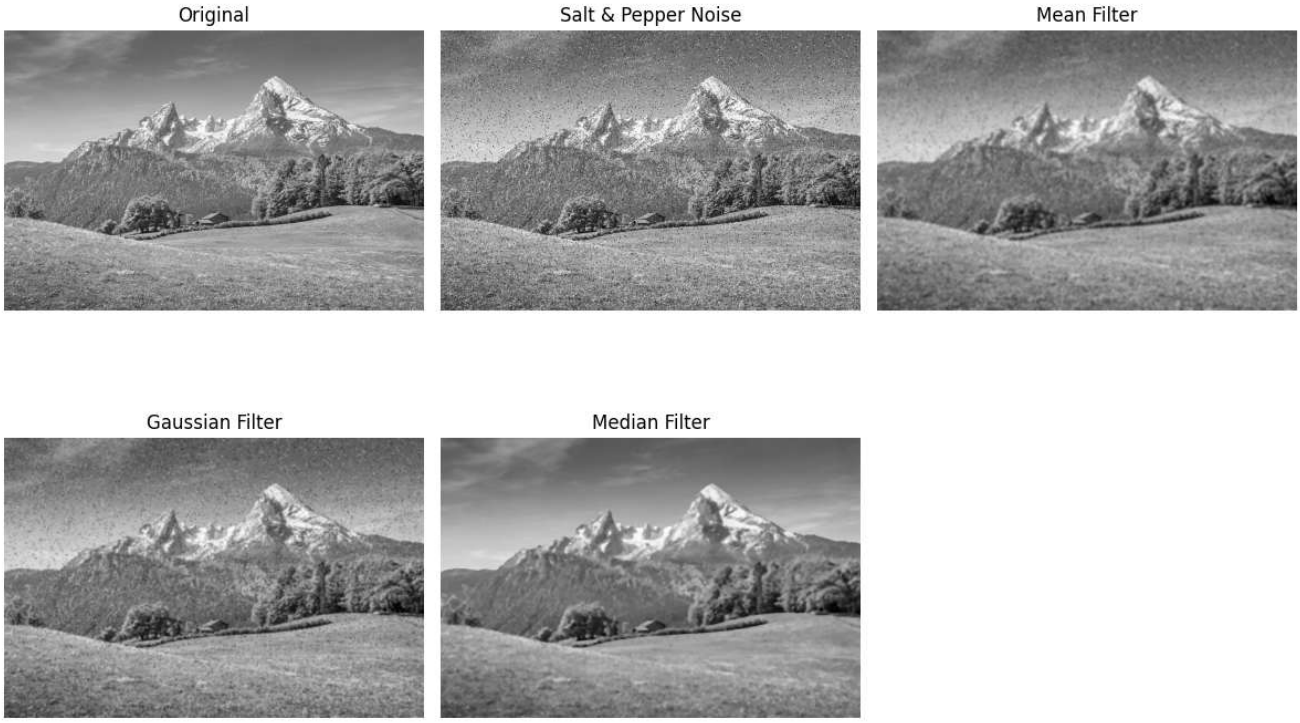
plt.subplot(2, 3, 3)
plt.imshow(mean, cmap='gray')
plt.title("Mean Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
MSE Results:
Mean Filter     : 430.26756206437466
Gaussian Filter : 346.36769715717753
Median Filter   : 383.52287767248805
```



2.

```
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("/content/SCENERY.jpeg", 0)
# Laplacian high-pass filter
laplacian = cv2.Laplacian(img, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# Sobel filters
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

# Combined Sobel edge map
sobel = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Edge Map")
plt.axis("off")

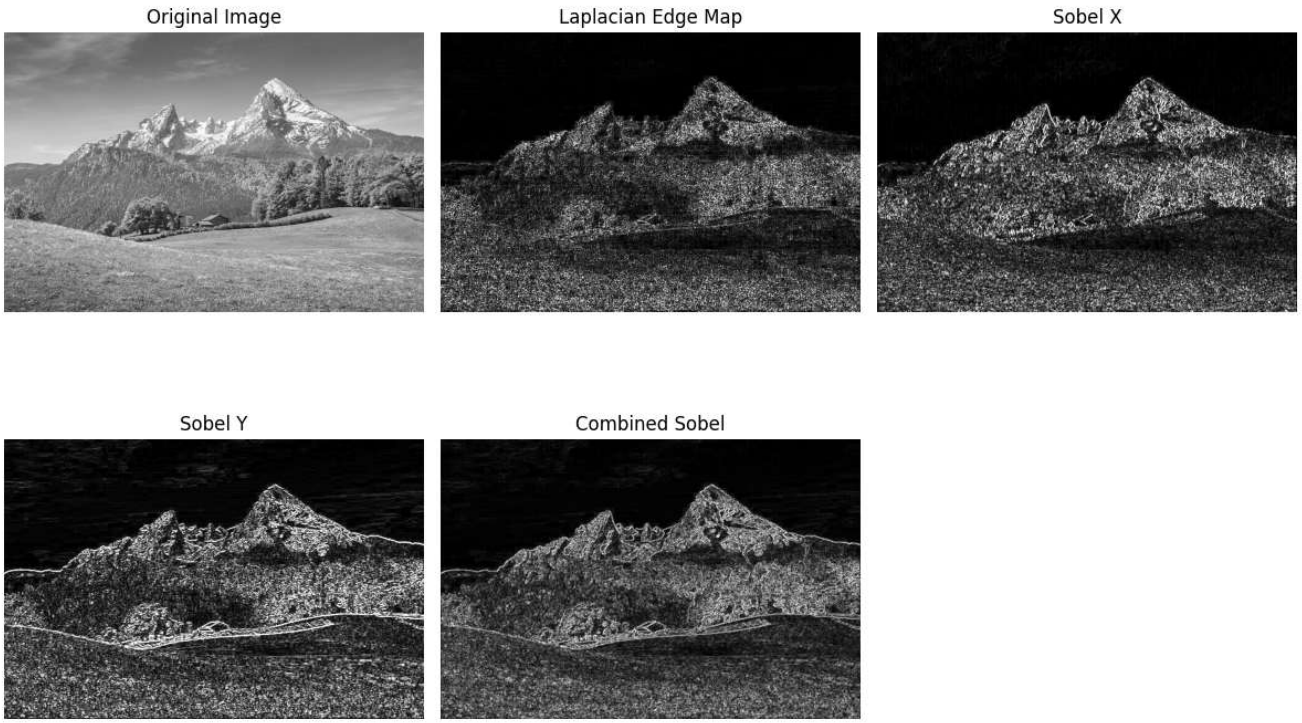
plt.subplot(2, 3, 3)
```

```
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(sobel, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.tight_layout()
plt.show()
```



3.

```
%%writefile streamlit_blur_app.py
import streamlit as st
import cv2
import numpy as np

st.set_page_config(layout="wide")

st.title("Interactive Image Blur Tool")

# Load image
# For demonstration, we'll use a fixed image. In a real app, you might use st.file_uploader
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    st.error(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    # Convert BGR to RGB for Streamlit display
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    st.sidebar.header("Filter Settings")
```

```
    # Filter selection
    filter_type = st.sidebar.selectbox(
        "Select Filter Type",
        ("Mean Blur", "Gaussian Blur", "Median Blur")
    )

    # Kernel size slider
    # Kernel size must be odd for most OpenCV filters
    kernel_size = st.sidebar.slider(
        "Select Kernel Size (odd number)",
        min_value=3, max_value=21, value=5, step=2
    )

    st.write(f"Applying **{filter_type}** with kernel size **{kernel_size}x{kernel_size}**")

    # Apply selected filter
    filtered_image = original_image.copy()
    if filter_type == "Mean Blur":
        filtered_image = cv2.blur(original_image, (kernel_size, kernel_size))
    elif filter_type == "Gaussian Blur":
        # SigmaX and SigmaY are often set to 0 to be calculated from kernel_size
        filtered_image = cv2.GaussianBlur(original_image, (kernel_size, kernel_size), 0)
    elif filter_type == "Median Blur":
        filtered_image = cv2.medianBlur(original_image, kernel_size)

    # Convert filtered image BGR to RGB for Streamlit display
    filtered_image_rgb = cv2.cvtColor(filtered_image, cv2.COLOR_BGR2RGB)

    # Display images side-by-side
    col1, col2 = st.columns(2)
    with col1:
        st.subheader("Original Image")
        st.image(original_image_rgb, caption="Original", use_column_width=True)
    with col2:
        st.subheader(f"Filtered Image ({filter_type})")
        st.image(filtered_image_rgb, caption=f"{filter_type} with {kernel_size}x{kernel_size} kernel", use_column_width=True)
```

```
Writing streamlit_blur_app.py
```

To run this Streamlit app, execute the cell below. A public URL will be provided to access the app in your browser.

```
!streamlit run streamlit_blur_app.py &>/dev/null&
```

4.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
# Using the same image as previous examples for consistency
img = cv2.imread("/content/SCENERY.jpeg", cv2.IMREAD_COLOR)

if img is None:
    print("Error: Could not load image. Please check the path.")
else:
    # Convert BGR to RGB for matplotlib display
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Unsharp Masking Parameters
    # These values can be adjusted to control the sharpening effect
    gaussian_radius = 5  # Determines the size of the blur filter
    amount = 1.5         # Controls the strength of the sharpening
    threshold = 0        # Pixels with difference below this threshold are not sharpened (optional)

    # 1. Apply Gaussian Blur to create a blurred version
    blurred_img = cv2.GaussianBlur(img, (0, 0), gaussian_radius) # SigmaX is calculated from radius if 0

    # 2. Calculate the unsharp mask (difference between original and blurred)
    unsharp_mask = cv2.addWeighted(img, 1.0 + amount, blurred_img, -amount, 0)

    # 3. Apply thresholding (optional, to avoid sharpening noise)
```

```
    # For simplicity, we are not applying thresholding in this example but it could be added
    sharpened_img = unsharp_mask

    # Convert sharpened image BGR to RGB for matplotlib display
    sharpened_img_rgb = cv2.cvtColor(sharpened_img, cv2.COLOR_BGR2RGB)

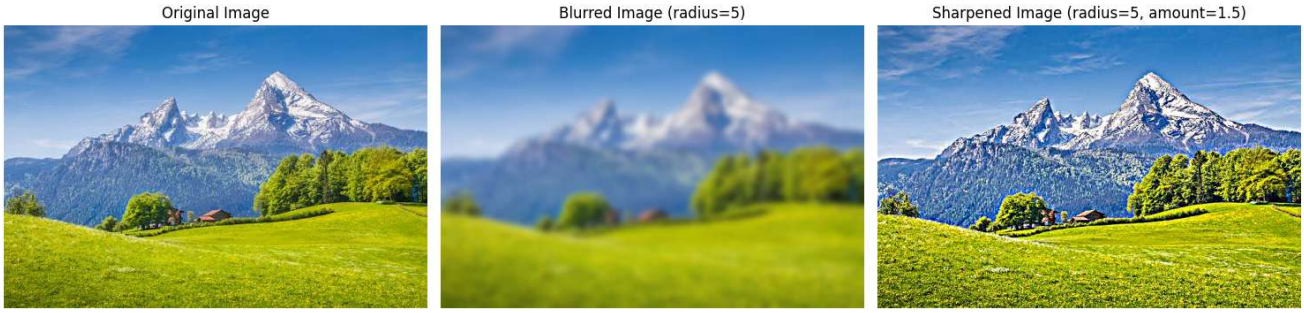
    # Display results
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(img_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(cv2.cvtColor(blurred_img, cv2.COLOR_BGR2RGB))
    plt.title(f"Blurred Image (radius={gaussian_radius})")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(sharpened_img_rgb)
    plt.title(f"Sharpened Image (radius={gaussian_radius}, amount={amount})")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
```



5.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    # Convert BGR to RGB for Matplotlib display
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    # --- Define your custom kernel here ---
    # Examples:
    # Identity kernel:
    # kernel_definition = [
    #     [0, 0, 0],
    #     [0, 1, 0],
    #     [0, 0, 0]
    # ]
    # Blur (Mean 3x3):
    # kernel_definition = [
    #     [1/9, 1/9, 1/9],
    #     [1/9, 1/9, 1/9],
    #     [1/9, 1/9, 1/9]
    # ]
```

```
# Sharpen:
# kernel_definition = [
#     [0, -1, 0],
#     [-1, 5, -1],
#     [0, -1, 0]
# ]
# Edge Detection (Laplacian):
# kernel_definition = [
#     [0, 1, 0],
#     [1, -4, 1],
#     [0, 1, 0]
# ]
# Emboss:
kernel_definition = [
    [-2, -1, 0],
    [-1, 1, 1],
    [0, 1, 2]
]

try:
    custom_kernel = np.array(kernel_definition, dtype=np.float32)

    if custom_kernel.ndim != 2 or custom_kernel.shape[0] == 0 or custom_kernel.shape[1] == 0:
        raise ValueError("Kernel must be a non-empty 2D array.")

    print("--- Custom Kernel Applied ---")
    print("Kernel shape:", custom_kernel.shape)
    print("Kernel values:\n", custom_kernel)

    # Apply convolution using cv2.filter2D
    # ddepth = -1 means the output image will have the same depth as the input
    processed_image = cv2.filter2D(original_image, -1, custom_kernel)

    # Convert processed image BGR to RGB for Matplotlib display
    processed_image_rgb = cv2.cvtColor(processed_image, cv2.COLOR_BGR2RGB)

    # Display images
    plt.figure(figsize=(12, 6))

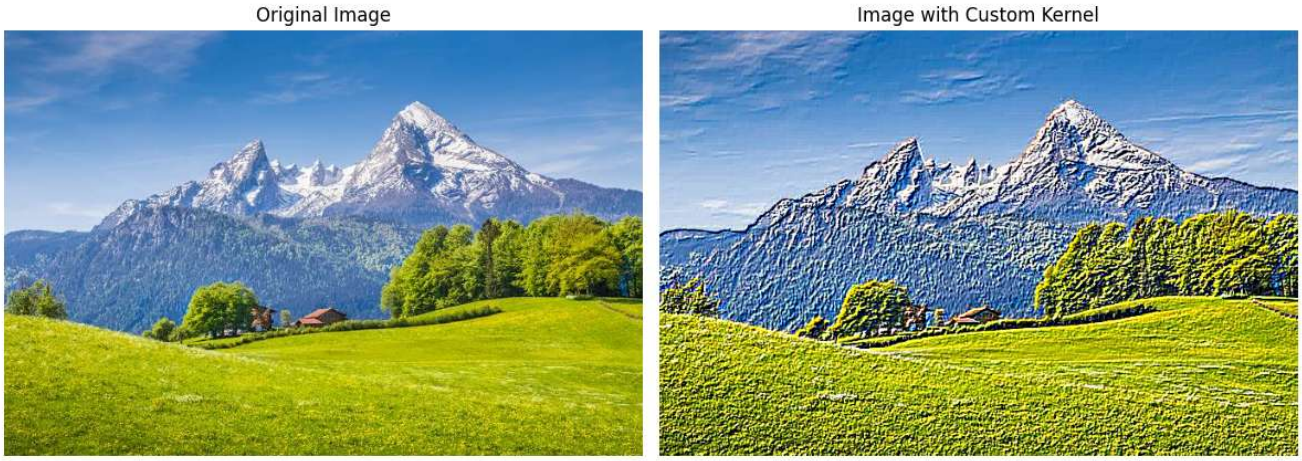
    plt.subplot(1, 2, 1)
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(processed_image_rgb)
    plt.title("Image with Custom Kernel")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

except ValueError as e:
    print(f"Error in kernel definition: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

```
--- Custom Kernel Applied ---
Kernel shape: (3, 3)
Kernel values:
 [[-2. -1.  0.]
 [-1.  1.  1.]
 [ 0.  1.  2.]]
```



6.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image (using the same image path as previous examples)
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    # Convert BGR to RGB for Matplotlib display
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    # --- Gaussian Blur Parameters ---
    gaussian_kernel_size = 5  # Must be an odd number (e.g., 3, 5, 7, ...)

    # --- Bilateral Filter Parameters ---
    d = 9             # Diameter of each pixel neighborhood that is used during filtering.
                      # A larger diameter means more pixels will be included in the filtering process,
                      # leading to stronger smoothing. (Odd values are common, e.g., 9, 15, 25).
    sigma_color = 75  # Filter sigma in the color space. A larger value means that more colors
                      # will be considered similar, leading to larger areas being smoothed together.
    sigma_space = 75  # Filter sigma in the coordinate space. A larger value means that farther pixels
                      # will influence the filtering, as long as their colors are close enough.

    print(f"Applying Gaussian Blur with Kernel Size: {gaussian_kernel_size}x{gaussian_kernel_size}")
    print(f"Applying Bilateral Filter with d={d}, sigmaColor={sigma_color}, sigmaSpace={sigma_space}")

    # Apply Gaussian Blur
    gaussian_blurred_image = cv2.GaussianBlur(original_image, (gaussian_kernel_size, gaussian_kernel_size), 0)

    # Apply Bilateral Filter
    bilateral_filtered_image = cv2.bilateralFilter(original_image, d, sigma_color, sigma_space)

    # Convert filtered images BGR to RGB for Matplotlib display
    gaussian_blurred_image_rgb = cv2.cvtColor(gaussian_blurred_image, cv2.COLOR_BGR2RGB)
    bilateral_filtered_image_rgb = cv2.cvtColor(bilateral_filtered_image, cv2.COLOR_BGR2RGB)

    # Display images
    plt.figure(figsize=(18, 6))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image_rgb)
```

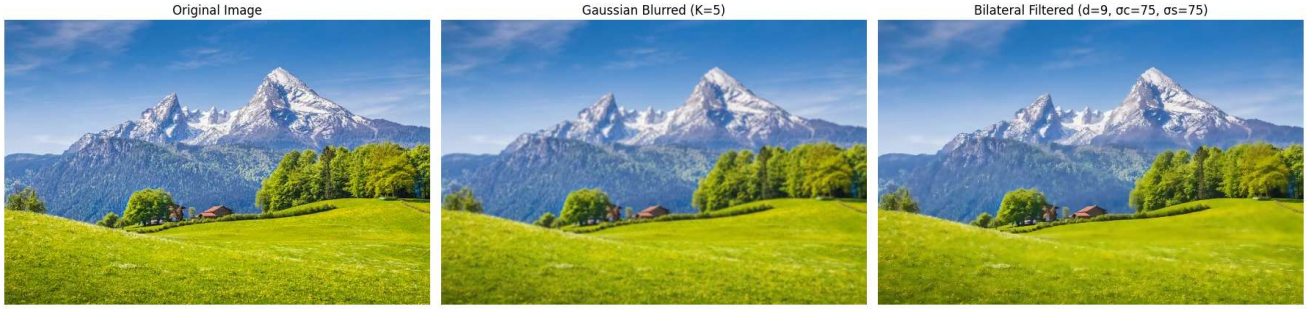
```
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(gaussian_blurred_image_rgb)
    plt.title(f"Gaussian Blurred (K={gaussian_kernel_size})")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(bilateral_filtered_image_rgb)
    plt.title(f"Bilateral Filtered (d={d}, σc={sigma_color}, σs={sigma_space})")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
```

```
Applying Gaussian Blur with Kernel Size: 5x5
Applying Bilateral Filter with d=9, sigmaColor=75, sigmaSpace=75
```



7.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.util import random_noise

# Load image
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    # Simulate noise (combining Gaussian and Salt-and-Pepper)
    # Convert to float for scikit-image's random_noise
    img_float = original_image_rgb / 255.0

    # Add Gaussian noise
    gaussian_noisy_image = random_noise(img_float, mode='gaussian', var=0.01) # var is variance

    # Add Salt-and-Pepper noise to the Gaussian noisy image
    sp_gaussian_noisy_image = random_noise(gaussian_noisy_image, mode='s&p', amount=0.05) # amount is fraction of pixels affecte

    # Convert back to uint8 for OpenCV and display
    noisy_image = (sp_gaussian_noisy_image * 255).astype(np.uint8)

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(noisy_image)
    plt.title("Simulated Noisy Image")
    plt.axis("off")
```

```
    plt.tight_layout()
    plt.show()
```




```
# Apply Median Filter first
median_filtered = cv2.medianBlur(noisy_image, 5) # Kernel size 5x5
```

```
# Apply Bilateral Filter on the output of the median filter
# Parameters: d (diameter), sigmaColor (color similarity), sigmaSpace (spatial proximity)
bilateral_filtered = cv2.bilateralFilter(median_filtered, d=9, sigmaColor=75, sigmaSpace=75)
```

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(noisy_image)
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(median_filtered)
plt.title("After Median Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(bilateral_filtered)
plt.title("After Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



8.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Load image
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    # --- Simulate Motion Blur ---
    # Create a motion blur kernel
    size = 15 # Length of the motion blur
    kernel_motion_blur = np.zeros((size, size), dtype=np.float32)
    # Blur in a diagonal direction (e.g., angle 45 degrees)
    for i in range(size):
        j = int(i * np.tan(np.deg2rad(45))) # Approximate diagonal
        if j < size:
            kernel_motion_blur[i, j] = 1
    kernel_motion_blur = kernel_motion_blur / np.sum(kernel_motion_blur)

    # Apply the motion blur kernel
    motion_blurred_image = cv2.filter2D(original_image, -1, kernel_motion_blur)
    motion_blurred_image_rgb = cv2.cvtColor(motion_blurred_image, cv2.COLOR_BGR2RGB)

    # Display original and motion blurred images
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(motion_blurred_image_rgb)
    plt.title("Simulated Motion Blurred Image")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```




```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Assuming `motion_blurred_image` is available from the previous cell
if 'motion_blurred_image' in locals() and motion_blurred_image is not None:

    # --- Unsharp Masking Parameters ---
    gaussian_radius = 5  # Determines the size of the blur filter for the mask
    amount = 1.5         # Controls the strength of the sharpening (higher = more sharp)

    print(f"Applying Unsharp Masking with Gaussian Radius: {gaussian_radius}, Amount: {amount}")

    # 1. Apply Gaussian Blur to the motion blurred image to create the 'blur' for the mask
    blurred_for_mask = cv2.GaussianBlur(motion_blurred_image, (0, 0), gaussian_radius)

    # 2. Calculate the unsharp mask (difference between motion blurred and its blurred version)
    # Formula: Sharpened = Original + amount * (Original - Blurred)
```

```
    # Which can be rewritten as: Sharpened = (1 + amount) * Original - amount * Blurred
    sharpened_image = cv2.addWeighted(motion_blurred_image, 1.0 + amount, blurred_for_mask, -amount, 0)

    sharpened_image_rgb = cv2.cvtColor(sharpened_image, cv2.COLOR_BGR2RGB)

    # Display results
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(motion_blurred_image_rgb)
    plt.title("Motion Blurred Image")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(sharpened_image_rgb)
    plt.title(f"Sharpened Image (Unsharp Mask)")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
else:
    print("Motion blurred image not found. Please run the previous cell to simulate motion blur.")
```

```
Applying Unsharp Masking with Gaussian Radius: 5, Amount: 1.5
```



9.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image_path = "/content/SCENERY.jpeg"
original_image = cv2.imread(image_path, cv2.IMREAD_COLOR)

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    original_image_rgb = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    # --- Apply Bilateral Filter ---
    # Repeatedly apply bilateral filter to achieve a stronger color reduction
    # The parameters (d, sigmaColor, sigmaSpace) can be tuned.
    num_iterations = 7 # Number of times to apply the bilateral filter
    bilateral_filtered_image = original_image.copy()
    for _ in range(num_iterations):
        bilateral_filtered_image = cv2.bilateralFilter(bilateral_filtered_image, d=9, sigmaColor=75, sigmaSpace=75)

    bilateral_filtered_image_rgb = cv2.cvtColor(bilateral_filtered_image, cv2.COLOR_BGR2RGB)

    # Display results
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
```

```
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(bilateral_filtered_image_rgb)
    plt.title("Color Smoothed (Bilateral Filter)")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```




```
# Assuming `original_image` is available from the previous cell
if 'original_image' in locals() and original_image is not None:

    # Convert to grayscale
    gray_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2GRAY)

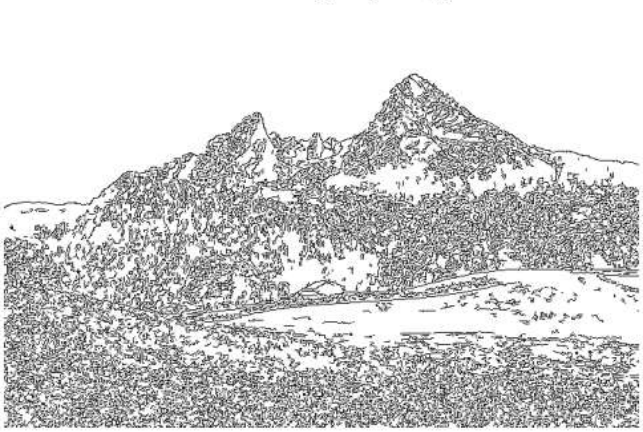
    # Apply Canny edge detection
    # Thresholds can be tuned for more/fewer edges
    edges = cv2.Canny(gray_image, 100, 200)

    # Invert edges to make them black (cartoon style)
    inverted_edges = cv2.bitwise_not(edges)

    # Display results
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(gray_image, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(inverted_edges, cmap='gray')
    plt.title("Inverted Edges (Canny)")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
else:
    print("Original image not found. Please run the first step of cartoonify generator.")
```






```
# Assuming `bilateral_filtered_image` and `inverted_edges` are available
if 'bilateral_filtered_image' in locals() and bilateral_filtered_image is not None and \
   'inverted_edges' in locals() and inverted_edges is not None:

    # Convert inverted_edges to a 3-channel image to combine with color image
    inverted_edges_color = cv2.cvtColor(inverted_edges, cv2.COLOR_GRAY2BGR)

    # Combine the color-smoothed image with the black outlines
    # The bitwise_and operation will 'paint' the black edges onto the smoothed image
    cartoon_image = cv2.bitwise_and(bilateral_filtered_image, inverted_edges_color)

    cartoon_image_rgb = cv2.cvtColor(cartoon_image, cv2.COLOR_BGR2RGB)

    # Display the final cartoonified image
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(original_image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(cartoon_image_rgb)
    plt.title("Cartoonified Image")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
else:
    print("Smoothed image or inverted edges not found. Please run previous steps.")
```





10.

```
!pip install scikit-image -q
# Install required library
!pip install scikit-image -q
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
import pandas as pd

# Upload image
uploaded = files.upload()

filename = list(uploaded.keys())[0]

# Read image
original = cv2.imread(filename)
original = cv2.cvtColor(original, cv2.COLOR_BGR2RGB)

# --------------------------------
# Add Salt & Pepper Noise
# --------------------------------

noisy = original.copy()

noise = np.random.rand(
    original.shape[0],
    original.shape[1]
)

noisy[noise < 0.02] = 0
noisy[noise > 0.98] = 255

# --------------------------------
# Apply Filters
# --------------------------------

mean_filter = cv2.blur(noisy, (5, 5))

gaussian_filter = cv2.GaussianBlur(
    noisy, (5, 5), 0
)

median_filter = cv2.medianBlur(
    noisy, 5
)

bilateral_filter = cv2.bilateralFilter(
    noisy, 9, 75, 75
)

# Store results
filters = {
    "Noisy": noisy,
    "Mean": mean_filter,
    "Gaussian": gaussian_filter,
    "Median": median_filter,
    "Bilateral": bilateral_filter
}

# --------------------------------
# Calculate PSNR & SSIM
# --------------------------------

results = []

for name, image in filters.items():

    psnr = peak_signal_noise_ratio(
        original,
        image,
        data_range=255
    )

    ssim = structural_similarity(
        original,
        image,
        channel_axis=2,
        data_range=255
    )
```

```
    results.append([
        name,
        round(psnr, 2),
        round(ssim, 4)
    ])

# Create table
df = pd.DataFrame(
    results,
    columns=["Filter", "PSNR (dB)", "SSIM"]
)

print("Performance Comparison:")
display(df)

# --------------------------------
# Display Images
# --------------------------------

plt.figure(figsize=(15, 8))

plt.subplot(2, 3, 1)
plt.imshow(original)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(noisy)
plt.title("Noisy")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(mean_filter)
plt.title("Mean Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(gaussian_filter)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(median_filter)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(bilateral_filter)
plt.title("Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

# --------------------------------
# Find Best Filters
# --------------------------------

best_psnr = df.loc[df["PSNR (dB)"].idxmax()]
best_ssim = df.loc[df["SSIM"].idxmax()]

print("Best PSNR:")
print(best_psnr)

print("\nBest SSIM:")
print(best_ssim)
```

Choose Files SCENERY.jpeg

SCENERY.jpeg(image/jpeg) - 67829 bytes, last modified: 9/14/2026 - 100% done

```
Saving SCENERY.jpeg to SCENERY (1).jpeg
Performance Comparison:
```

	Filter	PSNR (dB)	SSIM
0	Noisy	19.05	0.5155
1	Mean	22.20	0.4808
2	Gaussian	23.18	0.5562
3	Median	22.67	0.5777
4	Bilateral	18.79	0.4193

